## Activity 6: Exploring Sorting Runtimes:

| Array Size | Array Type | Sorting Algorithm | Execution Time (seconds) |
|---|---|---|---|
| 1,000 | Random | Merge Sort | 0.0549 |
| 1,000 | Random | Quick Sort (Left) | 0.0508 |
| 1,000 | Random | Quick Sort (Median-3) | 0.0488 |
| 1,000 | Random | Quick Sort (Random) | 0.0511 |
| 5,000 | Random | Merge Sort | 0.2518 |
| 5,000 | Random | Quick Sort (Left) | 0.2503 |
| 5,000 | Random | Quick Sort (Median-3) | 0.2386 |
| 5,000 | Random | Quick Sort (Random) | 0.2524 |
| 10,000 | Random | Merge Sort | 0.5285 |
| 10,000 | Random | Quick Sort (Left) | 0.5595 |
| 10,000 | Random | Quick Sort (Median-3) | 0.5156 |
| 10,000 | Random | Quick Sort (Random) | 0.5642 |
| 1,000 | Ascending | Merge Sort | 0.0064 |
| 1,000 | Ascending | Quick Sort (Left) | RecursionError |
| 1,000 | Ascending | Quick Sort (Median-3) | 0.0028 |
| 1,000 | Ascending | Quick Sort (Random) | 0.0031 |
| 5,000 | Ascending | Merge Sort | 0.0297 |
| 5,000 | Ascending | Quick Sort (Left) | RecursionError |
| 5,000 | Ascending | Quick Sort (Median-3) | 0.0164 |
| 5,000 | Ascending | Quick Sort (Random) | 0.0167 |
| 10,000 | Ascending | Merge Sort | 0.0562 |
| 10,000 | Ascending | Quick Sort (Left) | RecursionError |
| 10,000 | Ascending | Quick Sort (Median-3) | 0.0344 |
| 10,000 | Ascending | Quick Sort (Random) | 0.0354 |
| 1,000 | Descending | Merge Sort | 0.0051 |
| 1,000 | Descending | Quick Sort (Left) | RecursionError |
| 1,000 | Descending | Quick Sort (Median-3) | 0.0029 |
| 1,000 | Descending | Quick Sort (Random) | 0.0031 |
| 5,000 | Descending | Merge Sort | 0.0306 |
| 5,000 | Descending | Quick Sort (Left) | RecursionError |
| 5,000 | Descending | Quick Sort (Median-3) | 0.0164 |
| 5,000 | Descending | Quick Sort (Random) | 0.0167 |
| 10,000 | Descending | Merge Sort | 0.0620 |
| 10,000 | Descending | Quick Sort (Left) | RecursionError |
| 10,000 | Descending | Quick Sort (Median-3) | 0.0344 |
| 10,000 | Descending | Quick Sort (Random) | 0.0354 |

# Discussion of Results

The performance investigation of Merge Sort and the three Quick Sort variations reveals key characteristics of these advanced sorting algorithms. On average, with random data, all four algorithms perform similarly, scaling consistently with a time complexity of approximately

O(NlogN). The Quick Sort with Median-of-Three pivot selection showed a slight edge in this scenario, likely due to its ability to choose a consistently balanced pivot.

The most significant differences appear with pre-sorted data. Merge Sort demonstrates its remarkable consistency, performing exceptionally well on both ascending and descending arrays. Its "divide and conquer" strategy is unaffected by the initial order of the data, maintaining its O(NlogN) efficiency in all cases. In contrast, the basic Quick Sort using the left-most pivot fails on both ascending and descending data, resulting in a RecursionError. This perfectly illustrates its theoretical worst-case time complexity of O(N2), which occurs when the pivot choice consistently leads to unbalanced partitions.

Crucially, both the Median-of-Three and Random Pivot versions of Quick Sort completely solve this worst-case problem. By choosing a more strategic pivot, they avoid unbalanced partitions and perform just as efficiently on pre-sorted data as they do on random data, successfully maintaining their O(NlogN) average-case performance. This highlights the critical importance of pivot selection in the practical implementation of Quick Sort.